PureSVD for Time-Aware Movie Recommendation

Patrick Gomes de Oliveira

October 15, 2026

Abstract

We evaluate classical truncated singular-value decomposition for top-N MovieLens recommendation under a strict global temporal split. The system includes point-in-time features, strong non-neural baselines, reproducible Floci/AWS infrastructure, and an offline-to-online parity test.

1 Introduction

Movie recommendation tutorials commonly report a random or per-user leave-one-out split. We instead make the primary experiment respect global wall-clock time: no interaction after a cut-off can influence training. Our scope is deliberately classical machine learning. PureSVD is compared with shrunk item-item cosine and popularity; no neural network is used.

2 Dataset and Split

MovieLens ratings contain user, movie, rating, and Unix timestamp fields. Ratings of at least four define positive ranking interactions. An iterative five-core removes entities without enough evidence. If τ_1 and τ_2 are the 80th and 90th timestamp percentiles, respectively, training uses $t < \tau_1$, validation uses $\tau_1 \le t < \tau_2$, and test uses $t \ge \tau_2$. All transformations are fitted on training data only.

3 Feature Engineering

The declarative catalogue in `conf/features.yaml` covers user activity, rating statistics, movie popularity, age, genre count, recency, and time-decayed genre affinity. A prediction at time t uses only events with $t_{ui} < t$. Time-decayed affinity is

$$a_{uc}(t) = \sum_{j : t_{uj} < t} \mathbb{1}[c_j = c] \exp[-\lambda(t - t_{uj})].$$

Schema, future-injection, fit-scope, and training/serving parity tests enforce the contract. The generated feature catalogue is shown below.

Table 1: Engineered features. Every aggregate is computed point-in-time: only interactions strictly before the prediction cut-off contribute to its value.

Feature	Family	Transform
`user_activity_count`	user	`user_activity_count`
`user_mean_rating`	user	`user_mean_rating`
`user_tenure_days`	user	`user_tenure_days`
`user_recency_days`	context	`user_recency_days`
`user_genre_affinity`	text	`user_genre_affinity`
`item_popularity_rank`	item	`item_popularity_rank`
`item_trailing_popularity`	statistical	`item_trailing_popular`
`item_genre_tokens`	text	`item_genre_tokens`
`item_release_year`	item	`item_release_year`

4 Methodology

Let $X \in \{0,1\}^{|\mathcal{U}| \times |\mathcal{I}|}$ be the sparse training interaction matrix. PureSVD computes the rank-k approximation

$$X \approx U_k \Sigma_k V_k^\top$$

that minimizes $\|X - \hat{X}\|_F$ among rank-k matrices. The score for user u and item i is

$$\hat{y}_{ui} = (U_k)_{u:} \Sigma_k (V_k)_{:i}.$$

Items in the user's training history are removed before top-N selection. Unseen users receive the popularity ordering. Sparse Lanczos SVD avoids materializing the full decomposition.

The strongest baseline uses shrunk cosine:

$$s(i,j) = \frac{X_{:i}^\top X_{:j}}{\|X_{:i}\|_2 \|X_{:j}\|_2 + \beta}.$$

5 Experimental Setup

We report Recall@10, MRR@10, catalogue coverage, and

$$\mathrm{NDCG@}K = \frac{1}{|\mathcal{U}|} \sum_u \frac{\sum_{j=1}^{K} (2^{rel_{uj}} - 1) / \log_2(j+1)}{\mathrm{IDCG}_u@K}.$$

Candidates are the full catalogue minus training history. Hyperparameters are selected on validation only. Synthetic data is included for CI; real MovieLens data is downloaded on demand and is never committed.

6 Results

Table 2: Global temporal test results on the reproducible fixture.

Model	NDCG@10	Recall@10	MRR@10	Coverage
PureSVD	0.0848	0.1010	0.1546	0.2838
ItemKNN	0.0961	0.1209	0.1613	0.5292
Popularity	0.0863	0.0867	0.1841	0.0284

ItemKNN is strongest on the fixture. PureSVD substantially improves over the initial explicit-positive formulation but does not beat the tuned neighbourhood method. This negative result is retained: method complexity is not treated as evidence of superiority.

7 Deployment

Training writes factors and feature tables to S3 and publishes precomputed top-N lists to DynamoDB. Lambda performs a bounded key lookup. EventBridge schedules retraining and CloudWatch receives logs. The default Terraform profile targets Floci/LocalStack and therefore requires no paid AWS account.

8 Limitations and Conclusion

PureSVD is linear, explainable at the factor level, and inexpensive, but it does not model temporal drift inside the factorization and offers only a popularity fallback for cold users. Results on the fixture are not claims about MovieLens 20M. The project demonstrates a leakage-resistant, tested classical recommendation pipeline rather than a claim that SVD always wins.

References